NPCI DATA LAYER

Technical Architecture & Database Design

UPI · Unified Payments Interface

April 2026

Note: NPCI does not publicly disclose its complete internal stack. This document is based on confirmed engineer profiles, technical articles, and NPCI's open-source GitHub. Where specific tools are unconfirmed, this is stated clearly.

Contents

1. Overview 4
1.1 The Five-Layer Model 4
1.2 Key Design Principles 4
2. Layer 1 Redis Cluster (Active Transaction State) 5
2.1 What Redis Does 5
2.2 Data Stored in Redis 5
2.3 The 30-Second TTL Why It Matters 5
2.4 Why Redis and Not a Traditional Database 6
2.5 Redis Cluster Architecture 6
3. Layer 2 Cassandra (VPA Registry) 7
3.1 What Cassandra Does 7
3.2 VPA Registry Table Structure 7
3.3 How Cassandra Partitioning Works 7
3.4 Why Cassandra and Not MySQL/PostgreSQL 8
3.5 VPA Uniqueness How It Is Enforced 8
4. Layer 3 Kafka + Columnar Store (Append-Only Ledger) 9
4.1 What Kafka Does 9
4.2 Event Record Structure 9
4.3 The Hash Chain Tamper Evidence 9
4.4 Kafka Partitioning Strategy 10
4.5 Kafka Consumer Groups 10
4.6 Columnar Store (Hot Query Layer) 10
4.7 HDFS / Cold Archive 11
5. Layer 4 Apache Flink (Real-Time Fraud Scoring) 12
5.1 What Flink Does 12
5.2 Fraud Scoring Inputs 12
5.3 Fraud Score Thresholds 12
5.4 How Flink Maintains State 13
5.5 Flink vs Spark Streaming 13
6. Layer 5 Relational Database (Settlement) 14
6.1 What the Relational DB Does 14
6.2 Net Settlement Table 14

6.3 Settlement Flow 14
6.4 ACID Properties at the Right Layer 15
7. Complete Data Flow Alice to Bob 16
8. What NPCI Stores vs What Banks Store 17
8.1 NPCI Data (Routing Metadata) 17
8.2 Bank Data (Financial Records) 17
8.3 What You See in Each App 18
9. Source Verification 19
10. Glossary 20

1. Overview

The National Payments Corporation of India (NPCI) operates the UPI (Unified Payments Interface) network the world's largest real-time payment system, processing over 20 billion transactions per month as of August 2025. At this scale, no single database technology can handle all workloads. NPCI uses a polyglot persistence model multiple specialized database and streaming tools, each optimized for a specific job.

This document covers all five layers of NPCI's data architecture, the tools used at each layer, what data is stored, why that tool was chosen, and how the layers connect to each other.

1.1 The Five-Layer Model

Layer	Tool	Purpose
1 Active state	Redis Cluster	In-flight transaction state, deduplication, VPA cache
2 VPA registry	Cassandra	VPA to account number mapping 600M+ rows
3 Event ledger	Kafka + Columnar store	Append-only immutable transaction log
4 Fraud / risk	Apache Flink (or Spark)	Real-time stream processing and fraud scoring
5 Settlement	Relational DB (Oracle/PostgreSQL)	Net settlement positions for RTGS batch

1.2 Key Design Principles

- Polyglot persistence each problem solved by the best-fit tool
- Kafka as the single source of truth all other stores are projections of the Kafka log
- No single RDBMS relational DBs only used for small, structured settlement data
- Append-only ledger financial records are never updated, only appended
- TTL-based expiry active transaction state auto-expires after 30 seconds
- Privacy separation NPCI stores routing metadata only; account balances live at banks

2. Layer 1 Redis Cluster (Active Transaction State)

Confirmation status: CONFIRMED Listed by NPCI UPI engineer on LinkedIn profile alongside Kafka and Cassandra

2.1 What Redis Does

Redis is an in-memory key-value store. NPCI uses it for two critical jobs that must complete in under 1 millisecond:

- Deduplication check has this transaction ID been seen before?
- Active transaction state what is the current status of this in-flight payment?
- VPA resolution cache recently resolved VPAs cached to avoid hitting Cassandra repeatedly
- Server stickiness txnID to server ID mapping to ensure the same server handles all steps of a transaction

2.2 Data Stored in Redis

Key pattern	Value stored
txn:{TXN_ID}	status, payer VPA, payee VPA, amount, TTL=30s
vpa:{alice@sbi}	IFSC, account number, bank code cached lookup
srv:{TXN_ID}	Server node ID for stickiness routing
dedup:{TXN_ID}	Boolean flag seen = true, TTL=30s

2.3 The 30-Second TTL Why It Matters

Every active transaction key in Redis has a 30-second TTL (Time To Live). This is a financial safety mechanism:

- If Alice's app never receives a response, the transaction dies at 30s and is marked TIMEOUT
- If a debit happened but credit never confirmed within 30s, a compensating reversal fires automatically
- This prevents money being permanently stuck in a DEBIT_SUCCESS / CREDIT_PENDING state
- After 30s, the key is evicted Redis frees memory and the Kafka ledger becomes the authoritative record

2.4 Why Redis and Not a Traditional Database

At 10,000+ transactions per second, the deduplication check must complete in under 1ms. A disk-based database (PostgreSQL, MySQL) cannot achieve this at UPI scale. Redis stores everything in RAM and serves reads in microseconds. Redis Cluster distributes keys across many nodes, so the cluster scales horizontally as transaction volume grows.

2.5 Redis Cluster Architecture

Property	Detail
Deployment	Redis Cluster mode multiple master + replica nodes
Sharding	Keys distributed by hash slot across nodes
Replication	Each master has replica(s) automatic failover
Persistence	AOF (Append Only File) for durability on restart
Eviction policy	TTL-based expired keys auto-evicted from memory
Typical latency	< 1ms for GET/SET operations within data center

3. Layer 2 Cassandra (VPA Registry)

Confirmation status: CONFIRMED Listed by NPCI UPI engineer on LinkedIn profile alongside Redis and Kafka

3.1 What Cassandra Does

Cassandra is a distributed wide-column key-value store. At NPCI it serves one primary purpose: the VPA (Virtual Payment Address) registry. When Alice types bob@hdfc, NPCI must resolve this to Bob's real bank account number and IFSC code in under 5ms. With 600 million+ registered UPI IDs in India, this requires a database that can do fast point lookups at massive scale without a single point of failure.

3.2 VPA Registry Table Structure

Column	Example value
vpa (partition key)	alice@sbi
account_no	0091234567
ifsc	SBIN0001234
bank_name	State Bank of India
bank_code	SBI
psp_handle	sbi
registered_at	2023-08-14 10:22:01
status	ACTIVE

3.3 How Cassandra Partitioning Works

The VPA column is the partition key. Cassandra runs the VPA through a Murmur3 hash function to produce a token. The token determines which node in the cluster stores that row. This means:

- Any node can answer a VPA lookup without coordinating with other nodes
- No single node is a bottleneck reads are distributed across the entire cluster
- alice@sbi and bob@hdfc likely live on completely different nodes
- Adding more nodes automatically re-balances the data

```
hash("alice@sbi")  ->  token 4,611,686,018  ->  Node 3
```

```
hash("bob@hdfc")   ->  token 7,234,901,124  ->  Node 7
hash("raj@paytm")  ->  token 1,108,378,562  ->  Node 1
```

3.4 Why Cassandra and Not MySQL/PostgreSQL

Requirement	Why Cassandra wins
600M+ rows	Cassandra scales horizontally to billions of rows across nodes
5ms read target	Single-partition reads return in ~3-5ms without joins
No SPOF allowed	Masterless architecture any node can serve reads/writes
Write-rarely pattern	VPA registrations are rare; reads dominate Cassandra optimises for this
Geographic spread	Multi-DC replication for disaster recovery across NPCI data centers

3.5 VPA Uniqueness How It Is Enforced

Cassandra itself does NOT enforce uniqueness last-write-wins is its default. Uniqueness is enforced at the application layer by NPCI's VPA registration service. Before writing a new VPA to Cassandra, the service queries whether the VPA exists. If it does, registration is rejected and the user is prompted to choose an alternative. This is identical to how Gmail checks username availability before account creation.

4. Layer 3 Kafka + Columnar Store (Append-Only Ledger)

Confirmation status: CONFIRMED Listed by NPCI UPI engineer on LinkedIn. Multiple technical sources confirm Kafka as UPI's event messaging system

4.1 What Kafka Does

Apache Kafka is a distributed event streaming platform. At NPCI it serves as the immutable append-only ledger for every transaction event. Every time a transaction changes state INITIATED, DEBIT_SUCCESS, CREDIT_SUCCESS, COMPLETED NPCI publishes that event to a Kafka topic. Events are never modified or deleted. This makes Kafka the permanent source of truth for the entire transaction history of UPI.

4.2 Event Record Structure

Field	Example value
txn_id	TXN_8821A_20260409
event_type	DEBIT_SUCCESS
payer_vpa	alice@sbi
payee_vpa	bob@hdfc
amount	500.00
currency	INR
issuer_bank	SBI
acquirer_bank	HDFC
bank_ref	SBI_REF_9931
timestamp	2026-04-09T14:02:01.989Z
seq	3
prev_hash	a3f9c2d1...

4.3 The Hash Chain Tamper Evidence

Each event record contains a prev_hash field the SHA-256 hash of the previous event for that transaction. This creates a cryptographic chain. If anyone retroactively modifies Event 2, the hash

stored in Event 3 no longer matches, and the tampering is immediately detectable. This is the same principle used in blockchain, applied to financial audit trails.

4.4 Kafka Partitioning Strategy

Kafka topics are partitioned by txn_id. This guarantees that all events for a single transaction (INITIATED, DEBIT_SUCCESS, CREDIT_SUCCESS, COMPLETED) always land on the same partition, in order. This is critical because:

- Event ordering is guaranteed per transaction without distributed locks
- The fraud engine can read all events for a transaction in sequence
- Settlement batch can reconstruct any transaction's full history from one partition

4.5 Kafka Consumer Groups

Consumer	Latency requirement	What it does with events
Hot columnar store	Seconds	Writes to Druid/Pinot for fast dispute queries
Flink fraud engine	Milliseconds	Computes rolling aggregates and fraud scores
Notification service	Seconds	Pushes success/failure alerts to PSP apps
Settlement batch	Hours	Aggregates net positions for RTGS
HDFS archiver	Minutes–hours	Writes Parquet files for long-term cold storage

Each consumer group maintains its own offset its position in the Kafka log. The fraud engine may be at offset 10,000,000 while the archiver is at 9,990,000. They are completely independent. If the notification service goes down for 10 minutes, it catches up from its last offset without affecting other consumers.

4.6 Columnar Store (Hot Query Layer)

Kafka alone is not queryable for ad-hoc lookups. A streaming consumer reads from Kafka and writes completed transaction events into a hot columnar store (Apache Druid or Apache Pinot). This store covers the last 90 days and is optimised for:

- Dispute resolution show all transactions for alice@sbi in the last 30 days
- Regulatory queries all HDFC transactions in Q1 2026
- Merchant analytics all payments to merchant@swiggy in the last 7 days

4.7 HDFS / Cold Archive

Periodically a batch job reads from Kafka and writes Parquet files to HDFS (or equivalent cold storage). Parquet is a compressed columnar binary format. Files are organised by partition key (bank_id, date). This enables:

- Multi-year audit queries replay 3 years of SBI transactions
- RBI regulatory reporting monthly/quarterly transaction summaries
- Machine learning training data historical patterns for fraud model retraining

5. Layer 4 Apache Flink (Real-Time Fraud Scoring)

Confirmation status: INFERRED Flink or Apache Spark Streaming are cited in multiple independent technical articles as the likely stream processor. NPCI has not officially confirmed which tool is used.

5.1 What Flink Does

Apache Flink is a distributed stream processing framework. At NPCI it processes every transaction event in real time, maintaining per-user rolling aggregates and computing a fraud risk score before the PIN screen is shown to the payer. This means fraud scoring happens before any money moves at around t=20ms after the payment request arrives.

5.2 Fraud Scoring Inputs

Signal	What it detects
txn_count_last_1_min	Velocity attack user sending many payments rapidly
amount_last_1_min	Large amounts in short window unusual pattern
amount_last_1_hour	Daily limit abuse attempts
unique_payees_last_1_hour	Fan-out attack sending small amounts to many new accounts
new_payee_flag	First time sending to this VPA elevated risk
device_id_match	Payment from unregistered device
geo_anomaly	Payment from unexpected location
time_of_day	3am transactions statistically higher fraud rate

5.3 Fraud Score Thresholds

Score range	Action taken
0 – 40	Low risk proceed to PIN screen immediately
41 – 79	Medium risk proceed but flag for post-transaction review
80 – 100	High risk step-up challenge (additional OTP + PIN) or block

5.4 How Flink Maintains State

Flink uses tumbling windows and sliding windows over the Kafka event stream. Per-user aggregates (txn count, amount sum) are stored in RocksDB an embedded key-value store inside each Flink task manager. This avoids external database calls during score computation. State is checkpointed to HDFS periodically for fault tolerance. If a Flink node crashes, it restores from the last checkpoint and replays events from Kafka.

5.5 Flink vs Spark Streaming

Dimension	Flink vs Spark
Processing model	Flink: true streaming. Spark: micro-batch (slightly higher latency)
Latency	Flink: ~5–20ms. Spark Streaming: ~100–500ms
State management	Both use RocksDB; Flink has richer state APIs
UPI fit	Either works Flink preferred for sub-50ms scoring requirement
NPCI confirmation	Neither confirmed by NPCI publicly both are valid candidates

6. Layer 5 Relational Database (Settlement)

Confirmation status: INFERRED Oracle and PostgreSQL are standard choices for financial settlement systems. NPCI has not publicly disclosed the specific RDBMS used.

6.1 What the Relational DB Does

The relational database at NPCI is NOT used for transaction processing or real-time lookups. It serves a single, specific purpose: storing net settlement positions between banks for the RTGS batch that runs 6 times daily. It is a cold-path store updated in batches, not in real time.

6.2 Net Settlement Table

payer_bank	payee_bank	net_amount
SBI	HDFC	₹4,82,31,500
HDFC	SBI	₹3,91,12,000
ICICI	HDFC	₹1,23,45,000
AXIS	SBI	₹2,10,88,000

Instead of sending one RTGS instruction to RBI per transaction (billions per day), NPCI nets all transactions between each bank pair and sends a single instruction per pair per batch window. This reduces RBI's RTGS load from billions of messages to a few hundred.

6.3 Settlement Flow

Step	What happens
1. Events accumulate	Kafka receives COMPLETED events for all transactions
2. Batch job runs	Settlement service reads from Kafka, aggregates by bank pair
3. Net computed	SBI paid HDFC ₹10Cr, HDFC paid SBI ₹8Cr net: SBI owes HDFC ₹2Cr
4. Written to RDBMS	Net positions stored in settlement table with batch ID and timestamp
5. RTGS instruction	NPCI sends one RTGS message per net position to RBI
6. Final settlement	RBI's RTGS moves money between banks' reserve accounts

6.4 ACID Properties at the Right Layer

The relational database's ACID properties are important here because the settlement figures must be mathematically exact. A small rounding error in the net position calculation would create a discrepancy in the interbank settlement that could take days to reconcile. PostgreSQL or Oracle's ACID guarantees ensure:

- Atomicity the entire batch of net positions is written or none of it is
- Consistency total debits always equal total credits across all bank pairs
- Isolation concurrent settlement batch runs cannot corrupt each other
- Durability once settlement is committed, power failure cannot lose it

7. Complete Data Flow Alice to Bob

The following table traces exactly what happens at each data layer when Alice (alice@sbi) sends 500 to Bob (bob@hdfc).

Time	Action	Data layer touched
t=0ms	Alice taps Pay in app	None client side only
t=5ms	PSP server forwards to NPCI switch	None network hop
t=6ms	NPCI generates TXN_8821A	In-memory only no DB write yet
t=7ms	Redis dedup check	Redis: EXISTS txn:TXN_8821A? → No
t=7ms	Redis active state write	Redis: SET txn:TXN_8821A {INITIATED} TTL=30s
t=10ms	VPA resolution: alice@sbi	Cassandra: GET vpa=alice@sbi
t=10ms	VPA resolution: bob@hdfc	Cassandra: GET vpa=bob@hdfc
t=20ms	Fraud score computed	Flink: reads Kafka aggregates, score=12/100
t=50ms	Kafka event published	Kafka: INITIATED event appended
t=200ms	PIN auth request to SBI HSM	SBI HSM (bank side not NPCI)
t=400ms	SBI CBS ACID debit	SBI Core Banking: debit 500 from alice
t=410ms	Redis status updated	Redis: status=DEBIT_SUCCESS
t=410ms	Kafka event published	Kafka: DEBIT_SUCCESS event appended
t=600ms	HDFC CBS ACID credit	HDFC Core Banking: credit ₹500 to bob
t=610ms	Kafka event published	Kafka: CREDIT_SUCCESS event appended
t=800ms	Kafka final event	Kafka: COMPLETED event appended
t=800ms	Redis key expires	Redis: TXN_8821A evicted (job done)
t=1000ms	Notification sent	Kafka consumer → PSP notification service
t+4hrs	Settlement batch	RDBMS: SBI→HDFC net position updated
t+4hrs	RTGS instruction	RBI RTGS: single interbank settlement

8. What NPCI Stores vs What Banks Store

A critical architectural decision in UPI is the strict separation between NPCI's routing metadata and the banks' financial records. NPCI is a routing and coordination infrastructure it is not permitted to hold account balances or personal financial history.

8.1 NPCI Data (Routing Metadata)

Data element	Stored by NPCI?
Transaction ID (TXN_8821A)	Yes primary key across all NPCI stores
Payer VPA (alice@sbi)	Yes routing identifier
Payee VPA (bob@hdfc)	Yes routing identifier
Amount (₹500)	Yes needed for settlement netting
Transaction status	Yes in Redis (active) and Kafka (permanent)
Fraud score	Yes in Flink aggregates
Bank codes (SBI, HDFC)	Yes for routing
Alice's account number	No only resolved from Cassandra for routing
Alice's account balance	No never stored by NPCI
Alice's KYC / Aadhaar	No held by bank and UIDAI
Alice's transaction history	Routing log only not full financial statement

8.2 Bank Data (Financial Records)

Each bank's Core Banking System (CBS) maintains the authoritative financial record for its own customers. This is what appears in your bank statement, passbook, and banking app.

Data element	Stored by bank
Account number	SBI stores Alice's, HDFC stores Bob's
Running balance	Before and after every transaction
Transaction narration	UPI/TXN_8821A/bob@hdfc the statement line
Bank reference number	SBI_REF_9931 internal bank reference

Full KYC	Name, Aadhaar, PAN, address
Statement history	Every debit and credit for the lifetime of the account

8.3 What You See in Each App

App / interface	Data source
PhonePe / GPay transaction history	NPCI routing log txn ID, VPA, amount, status
BHIM transaction history	NPCI routing log (NPCI's own app)
SBI YONO transaction history	SBI CBS full debit/credit with balance
Bank passbook	SBI/HDFC CBS legally authoritative financial record
NPCI dispute portal	Both NPCI log + bank reference number

9. Source Verification

NPCI does not publish a complete internal technology stack document. The table below shows the confidence level for each tool mentioned in this document.

Tool	Confidence	Evidence
Redis	CONFIRMED	NPCI UPI engineer LinkedIn profile lists Redis explicitly
Kafka	CONFIRMED	NPCI UPI engineer LinkedIn profile; multiple engineering articles describe Kafka as UPI's event queue
Cassandra	CONFIRMED	Same NPCI engineer profile lists Cassandra alongside Redis and Kafka
Apache Flink	INFERRED	Multiple architecture articles cite Flink or Spark Streaming; no NPCI confirmation of which tool
HDFS / Parquet	INFERRED	Standard industry pattern for Kafka → cold archive; NPCI GitHub confirms open-source tools but does not name HDFS
Oracle / PostgreSQL	INFERRED	Standard for financial settlement systems; NPCI has not named their specific RDBMS
NPCI open source	CONFIRMED	NPCI GitHub (github.com/NPCI) states platform is built on open-source tools

Sources used:

- LinkedIn: Sachin Vishwakarma Senior Associate UPI Design and Development at NPCI (Java, Spring Boot, Kafka, Redis, Cassandra)
- Medium: 'Inside UPI: The Technology, Security, and Workflow of India's Payment Giant' Sai Kalyan, February 2025
- Medium: 'The Data Behind UPI: Understanding the Architecture of Real-Time Financial Pipelines' Ankit Gochhayat, June 2025
- GitHub: github.com/NPCI NPCI official open-source account
- Wikipedia: Unified Payments Interface transaction scale statistics (August 2025)

10. Glossary

Term	Definition
VPA	Virtual Payment Address UPI ID like alice@sbi, the human-readable alias for a bank account
NPCI	National Payments Corporation of India the body that owns and operates UPI
PSP	Payment Service Provider apps like PhonePe, GPay that provide a UPI interface
CBS	Core Banking System the bank's internal ledger software (Finacle, Temenos, BaNCS)
HSM	Hardware Security Module tamper-proof hardware device that validates UPI PIN
TTL	Time To Live how long a Redis key lives before auto-deletion
Kafka partition	A subset of a Kafka topic; all events with the same txn_id go to the same partition
Append-only	A write pattern where records are only added, never modified or deleted
Columnar store	A database that stores data column-by-column for fast analytical queries (vs row-by-row)
Parquet	A compressed columnar binary file format used for big data archival
RTGS	Real Time Gross Settlement RBI's system for large interbank money transfers
BBPS	Bharat Bill Payment System NPCI's layer for utility bill payments built on UPI rails
ACID	Atomicity, Consistency, Isolation, Durability properties of reliable database transactions
Polyglot persistence	Using multiple different database technologies, each for the job it is best suited for
Deduplication	Detecting and rejecting duplicate payment requests to prevent double charges
Netting	Aggregating many bilateral transactions into a single net amount for settlement efficiency